



Materia: Ciber física

Actividad: U4A5 REPORTE TÉCNICO
FINAL. Demo y
documentación

Nombre: Diego Alberto Alcaraz González

21030002

Carlos Diego Domínguez Lopez

21030006

Cinthia Garcia Garcia

22030009

Docente: Osbaldo Aragón Banderas

Fecha: 10/06/2026



Contenido

Introducción	3
Objetivo	3
Marco teórico.....	4
Node-RED.....	4
Gemelo digital	5
Desarrollo	6
Codigo en Python del gemelo digital	6
Código del seguidor de línea con telemetría.....	8
Dashboard	8
Flow de node-red.....	10
Tabla de características seguidor físico	10
Análisis de la gráfica de trayectoria.....	¡Error! Marcador no definido.
Configuración del gemelo digital	11
Conclusiones	24
Referencias.....	29



Introducción

EL reporte muestra y documenta el desarrollo del prototipo final de un gemelo digital aplicado a un robot seguidor de línea, integrando conceptos de Sistemas Ciberfísicos (CPS), Internet de las Cosas (IoT) e Inteligencia Artificial (IA). El sistema fue diseñado para capturar información en tiempo real del robot físico mediante sensores infrarrojos QTR, procesar los datos de navegación y transmitirlos a través del protocolo MQTT para su almacenamiento, monitoreo y análisis. La arquitectura implementada permite que el robot opere de forma autónoma utilizando un controlador PID encargado de corregir la trayectoria respecto a la línea de seguimiento. Durante la operación se registran variables relevantes como lecturas de sensores, error de posición, corrección aplicada por el controlador, velocidades de los motores y estados de navegación. Estos datos constituyen la base para la construcción del dataset utilizado en el entrenamiento del modelo de inteligencia artificial.

Como componente de análisis inteligente se empleó un modelo de aprendizaje supervisado tipo Random Forest, cuyo objetivo es identificar el estado operativo del robot a partir de las variables capturadas durante su funcionamiento. De esta manera, el gemelo digital no solo replica el comportamiento del sistema físico, sino que también incorpora capacidades de interpretación y diagnóstico mediante técnicas de inteligencia artificial. La integración de adquisición de datos, comunicación IoT, control PID y aprendizaje automático permite validar el funcionamiento del sistema CPS+IA, proporcionando una plataforma capaz de monitorear, analizar y representar digitalmente el comportamiento dinámico del robot seguidor de línea.



Objetivo

Demostrar y documentar el prototipo final del gemelo digital del robot seguidor de línea, integrando arquitectura, adquisición de datos, modelo, resultados, evidencias y lecciones aprendidas para validar el funcionamiento del sistema CPS+IA.

Descripción breve del problema resuelto

El robot seguidor de línea presentaba oscilaciones durante su recorrido debido a pequeñas variaciones en las lecturas de los sensores y a correcciones continuas realizadas por el controlador PID. Estas oscilaciones provocan movimientos inestables, pérdida de eficiencia en el seguimiento de la trayectoria y un mayor desgaste de los motores. Para abordar esta problemática, se desarrolló un sistema basado en un gemelo digital e inteligencia artificial capaz de analizar el comportamiento del robot a partir de datos reales de operación. Mediante la captura de telemetría, la construcción de un dataset y el entrenamiento de un modelo Random Forest, fue posible identificar una configuración óptima de zona muerta que reduce correcciones innecesarias y mejora la estabilidad del sistema.

La solución implementada permite evaluar diferentes configuraciones en un entorno de simulación antes de aplicarlas al robot físico, facilitando la toma de decisiones y validando el desempeño del sistema dentro de una arquitectura que integra Sistemas Ciberfísicos, IoT e Inteligencia Artificial.

. Marco teórico

Node-RED

La captura de datos se realizó utilizando Node-RED como plataforma de adquisición y monitoreo en tiempo real. El robot envió continuamente las lecturas de sus cuatro sensores analógicos durante todo el recorrido de la pista, las cuales fueron recibidas y organizadas en Node-RED. Posteriormente, estos datos fueron almacenados en un archivo CSV, permitiendo su uso en etapas posteriores de análisis. Este proceso



facilitó la visualización de los datos y su registro estructurado para su procesamiento en Python.

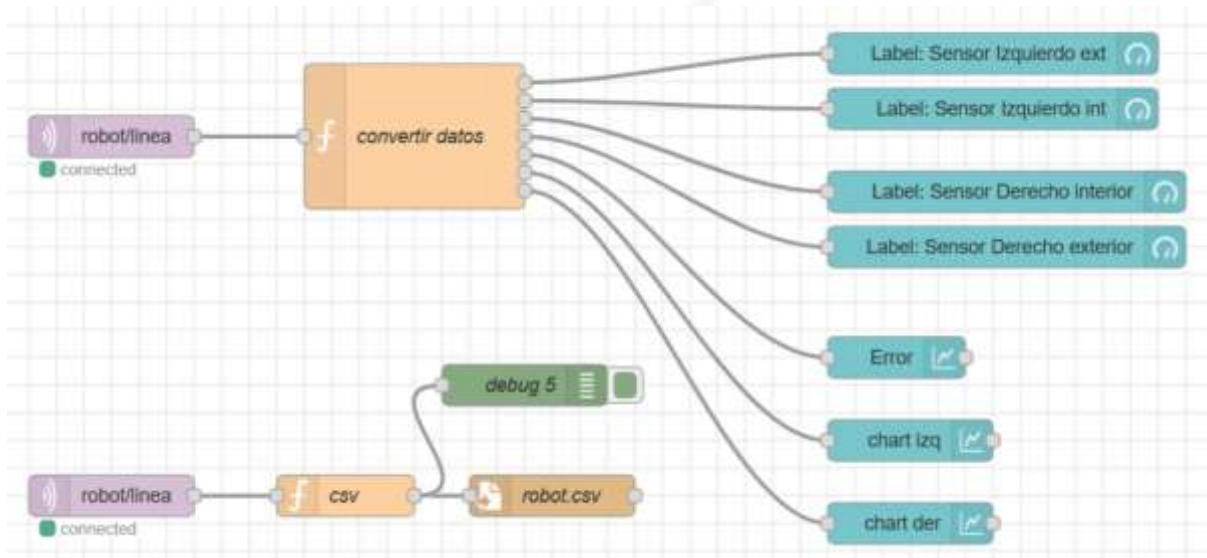


Figura 1: Nodos en Node-Red

Gemelo digital

El concepto de gemelo digital hace referencia a la representación virtual de un sistema físico, que permite simular su comportamiento bajo diferentes condiciones. En el contexto de esta práctica, el gemelo digital desarrollado en PyQt5 mediante Python permite modelar tanto el control como la dinámica del robot. Este tipo de herramientas facilita la experimentación, ya que permite realizar pruebas sin riesgo físico, ajustar parámetros de forma rápida y analizar el comportamiento del sistema antes de implementarlo en el mundo real. Además, permite comparar resultados entre simulación y realidad, identificando diferencias y mejorando la precisión del modelo.

En este caso el gemelo digital fue una modificación de un programa de Git-hub agregando la pista, las características de chasis, voltaje, motores y de PID, añadiendo también una zona muerta la cual es el objetivo a incorporar con ayuda de la clasificación pudiendo activar y desactivar la zona muerta con diferentes valores para así evaluar el desempeño.



Desarrollo

Código en Python del gemelo digital

```
6  v Aplicación de escritorio que simula un seguidor de línea con:
26  os.environ["QT_API"] = "PyQt5"
27  os.environ["MPLBACKEND"] = "Qt5Agg"
28
29  import sys
30  import json
31  import csv
32  import copy
33  import time
34  from collections import deque
35
36  import numpy as np
37
38  from PyQt5.QtCore import Qt, QTimer, pyqtSignal, QPointF, QRectF, QSize
39  from PyQt5.QtGui import (QPixmap, QPainter, QColor, QPen, QBrush, QFont,
40  | | | | | QPolygonF, QImage, QTransform, QIcon)
41  from PyQt5.QtWidgets import (
42  |   QApplication, QMainWindow, QWidget, QHBoxLayout, QVBoxLayout, QGridLayout,
43  |   QLabel, QPushButton, QDoubleSpinBox, QSpinBox, QSlider, QComboBox,
44  |   QFileDialog, QGroupBox, QTabWidget, QSplitter, QFrame, QSizePolicy,
45  |   QScrollArea, QToolTip, QStatusBar, QMessageBox, QStyleFactory,
46  | )
47
48  from PIL import Image, ImageDraw
49
50  import matplotlib
51  matplotlib.use("Qt5Agg", force=True)
52  from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas
```

Figura 2: Librerías usadas para el gemelo digital

El código en Python implementa un gemelo digital de un robot seguidor de línea que simula tanto su sistema de control como su comportamiento físico dentro de una pista virtual. A través de PyQt5 se construye una interfaz gráfica interactiva donde se visualiza el robot en tiempo real, su trayectoria y diferentes variables de operación. La simulación incluye una pista que puede ser generada automáticamente o cargada como imagen, sobre la cual actúan cuatro sensores virtuales que imitan el funcionamiento de sensores reflectivos reales, midiendo la intensidad de la línea mediante valores analógicos. Estas lecturas se procesan



mediante un algoritmo PID (principalmente PD), que calcula un error basado en la posición de la línea respecto al centro del robot, permitiendo ajustar dinámicamente la velocidad de cada motor para mantener la trayectoria.

Por otro lado, el programa no solo simula el control, sino también la física del robot, considerando aspectos como masa, fricción, torque de los motores y velocidad angular, lo que permite obtener un comportamiento más realista. En cada ciclo de simulación, el sistema actualiza las lecturas de sensores, calcula la corrección del controlador y aplica las fuerzas correspondientes al modelo físico para determinar el movimiento del robot. Además, se registran datos de telemetría como error, velocidad, PWM y posición, los cuales pueden visualizarse en gráficas en tiempo real o exportarse para análisis posterior, incluso comparándolos con datos reales obtenidos mediante herramientas como Node-RED. De esta manera, el gemelo digital se convierte en una herramienta clave para ajustar parámetros, analizar el desempeño y lograr que la simulación represente lo más fielmente posible el comportamiento del robot físico.



Código del seguidor de línea con telemetría

```
447     if(millis() - lastSend > 100){
448
449         String payload = "{";
450
451         payload += "\"time\":" + String(millis()) + ",";
452         payload += "\"test\":" + String(pruebaID) + ",";
453
454         payload += "\"s0\":" + String(v0) + ",";
455         payload += "\"s1\":" + String(v1) + ",";
456         payload += "\"s2\":" + String(v2) + ",";
457         payload += "\"s3\":" + String(v3) + ",";
458
459         payload += "\"error\":" + String(error) + ",";
460         payload += "\"corr\":" + String(correction) + ",";
461
462         payload += "\"left\":" + String(leftSpeed) + ",";
463         payload += "\"right\":" + String(rightSpeed) + ",";
464
465         payload += "\"kp\":" + String(Kp) + ",";
466         payload += "\"kd\":" + String(Kd) + ",";
467
468         payload += "\"estado\":" + estado + "\"";
469
470         payload += "}";
471
472         client.publish("robot/linea", payload.c_str());
473
474         lastSend = millis();
475     }
```

Ilustración 1: Parte de telemetría del seguidor

Dashboard

La figura muestra el dashboard desarrollado para el monitoreo en tiempo real del sistema de seguimiento ("Seguidor"), el cual integra información proveniente de sensores y gráficos de comportamiento.

En el panel izquierdo se presentan los valores de los sensores del lado izquierdo, diferenciando entre sensor exterior e interior. Estos indicadores se muestran



mediante visualizaciones circulares que permiten una lectura rápida de las magnitudes registradas, alcanzando valores de 832 unidades para el sensor exterior y 153 unidades para el interior.

En la sección central se incluyen dos gráficos de series temporales que representan la variación de los sensores izquierdo y derecho a lo largo del tiempo. Estas gráficas permiten identificar fluctuaciones, picos y tendencias en los datos, lo cual resulta fundamental para analizar el comportamiento dinámico del sistema. Adicionalmente, se incorpora un gráfico de error, el cual muestra la diferencia o desviación en el proceso de calibración, facilitando la evaluación del desempeño y la precisión del sistema.

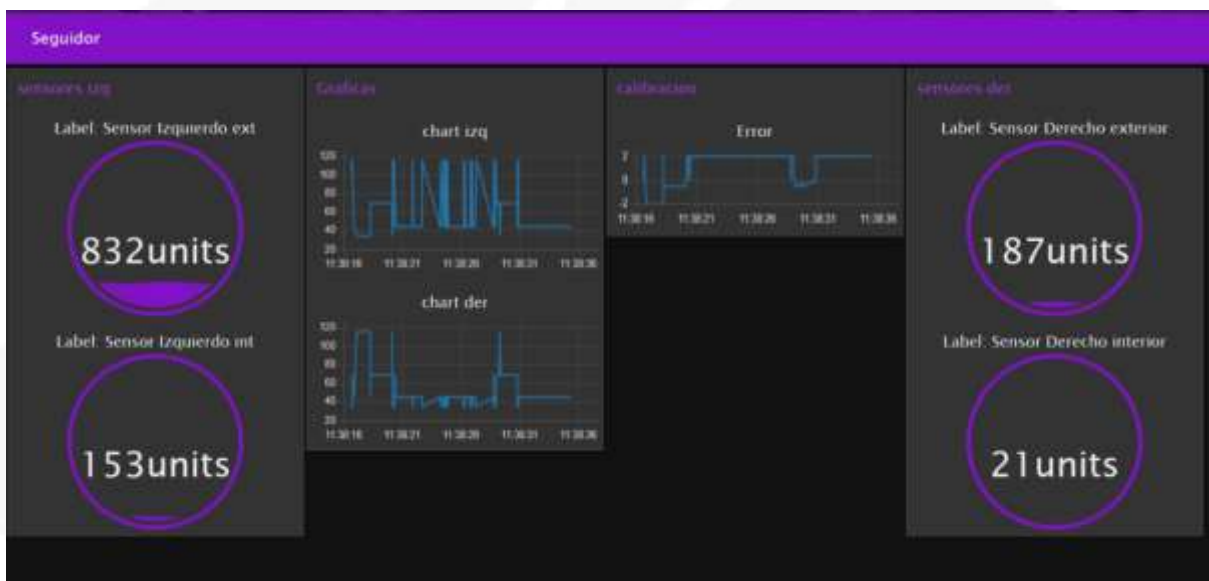


Figura 3: Dashboard en Node-red

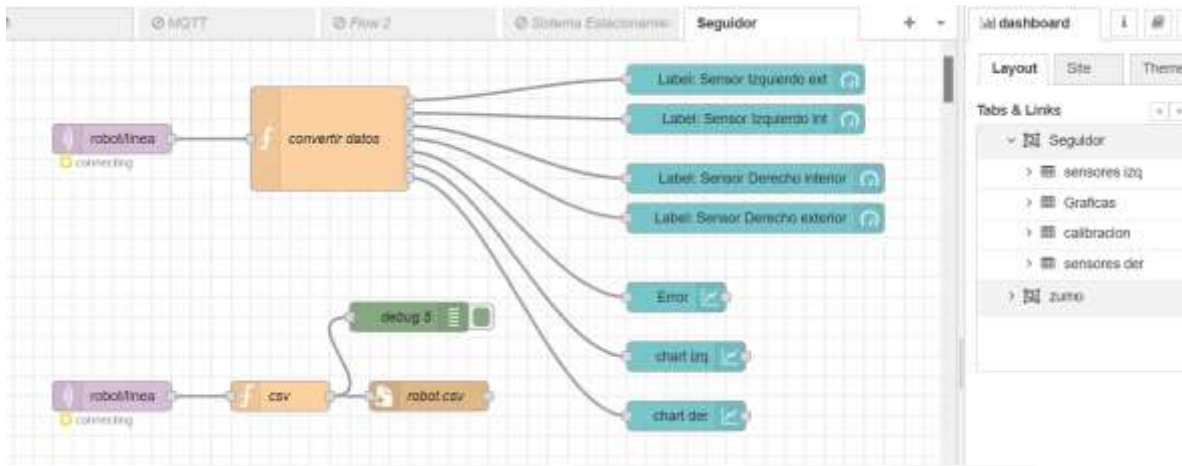


Figura 4: flow del sistema en node-red

Tabla 1: características seguidor físico

Largo	16.5 cm
Ancho	14.8 cm
Distancia entre sensores	1 cm
Distancia sensores del eje	12.5 cm
Diámetro de ruedas	16.5 cm
Peso	218 gramos
Motorreductores	48:1



Voltaje Inicial	9.12V
KP	0.20
KD	0.14

Parámetros de adquisición

Tabla 2; Parámetros de adquisición de datos

Variable	Descripción	Unidad
time	Marca temporal de la muestra	ms
test	Identificador de prueba	int
s0	Lectura del sensor izquierdo exterior	ADC
s1	Lectura del sensor izquierdo interior	ADC
s2	Lectura del sensor derecho interior	ADC
s3	Lectura del sensor derecho exterior	ADC
error	Error de seguimiento calculado por el controlador	Float
corr	Corrección aplicada por el controlador PID	Float
left	Velocidad o señal enviada al motor izquierdo	PWM
right	Velocidad o señal enviada al motor derecho	PWM
kp	Ganancia proporcional utilizada	Float



kd	Ganancia derivativa utilizada	Float
estado	Estado operativo del robot	String

Configuración del seguidor de línea físico y digital antes del entrenamiento

Tabla 3: Tabla comparativa gemelo digital y seguidor físico

Característica	Seguidor físico	Gemelo digital
Largo	16.5 cm	0.165 m (16.5cm)
Ancho	14.8 cm	0.147 m (14.7cm)
Distancia entre sensores	1 cm	0.0100.m (1cm)
Distancia sensores del eje	12.5 cm	.125 (12.5cm)
Diámetro de ruedas	16.8 cm	0.168m (16.8cm)
Peso	218 gramos	0.218 kg (218gr)
Motorreductores	48:1	330 rpm (revolución máxima de un motorreductor 48:1)
Voltaje Inicial	9.12V	9.20V
KP	0.03	0.03
KD	0.02	0.02
Zona Muerta	Desactivada	Desactivada

Tiempo de recorrido antes del entrenamiento

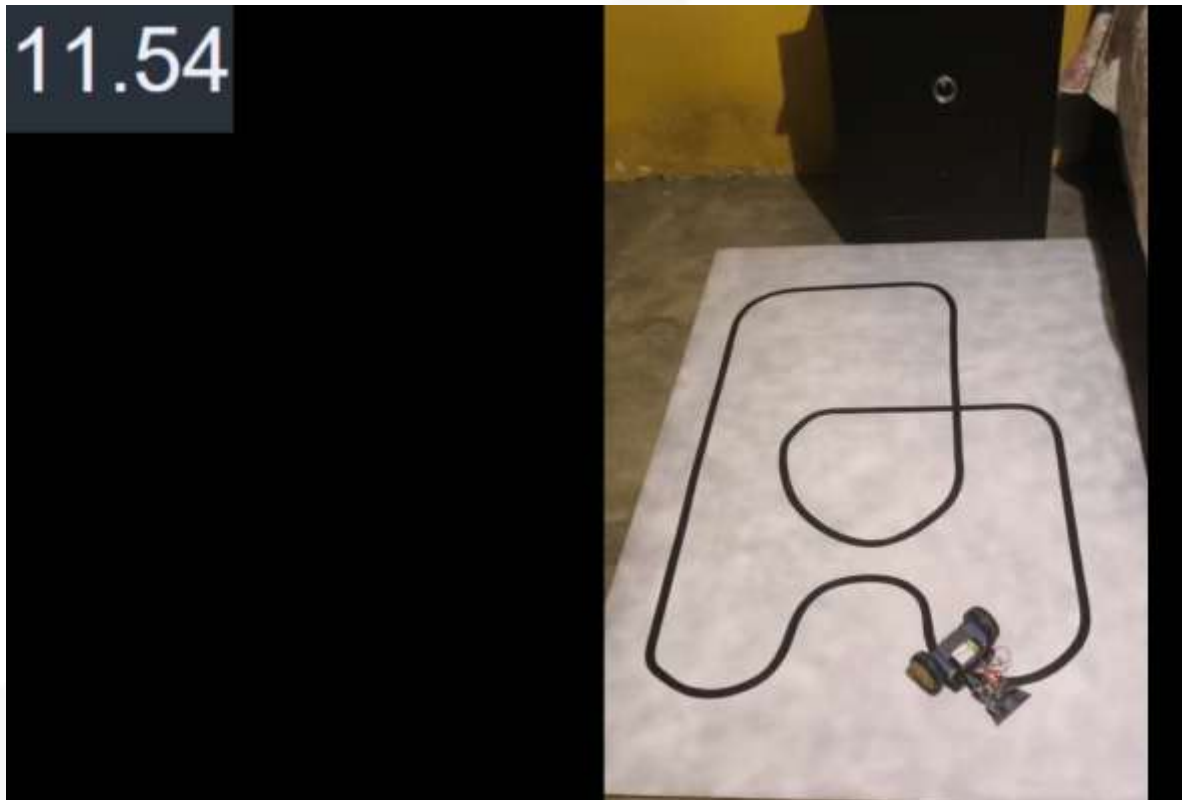


Figura 5: Tiempo de recorrido del seguidor antes del entrenamiento



Figura 6: Tiempo de recorrido del gemelo digital antes del entrenamiento



Procesado de datos del CSV

Tabla 4: Tabla de información general del dataset

```
=====
INFORMACION GENERAL DEL DATASET
=====
Filas: 586
Columnas: 13

Columnas:
['hora', 'test', 's0', 's1', 's2', 's3', 'error', 'corr', 'left', 'right', 'kp', 'kd', 'estado']

Primeras filas:
```

	hora	test	s0	s1	s2	s3	error	corr	left	right	kp	kd	estado
0	11:59:36	1	916	33	18	143	0.75	0.03	179	180	0.03	0.02	centrado
1	11:59:36	1	942	27	23	23	2.77	0.09	179	180	0.03	0.02	vuelta_derecha
2	11:59:36	1	55	855	843	24	0.88	0.02	179	180	0.03	0.02	desviado
3	11:59:36	1	36	43	597	38	0.00	0.00	-40	200	0.03	0.02	recuperacion
4	11:59:37	1	39	47	431	72	0.00	0.00	-40	200	0.03	0.02	recuperacion

Tabla 5: Tipos de datos del CSV

```
=====
TIPOS DE DATOS
=====
```

hora	str
test	int64
s0	int64
s1	int64
s2	int64
s3	int64
error	float64
corr	float64
left	int64
right	int64
kp	float64
kd	float64
estado	str
dtype:	object



Tabla 6: Valores faltantes del CSV

```
=====
VALORES FALTANTES
=====
```

	Variable	Valores Nulos
0	hora	0
1	test	0
2	s0	0
3	s1	0
4	s2	0
5	s3	0
6	error	0
7	corr	0
8	left	0
9	right	0
10	kp	0
11	kd	0
12	estado	0

Total de valores nulos: 0

Tabla 7: Valores registrados y estadísticas descriptivas

```
=====
REGISTROS DUPLICADOS
=====
Registros duplicados encontrados: 0
=====
ESTADÍSTICAS DESCRIPTIVAS
=====
```

	test	s0	s1	s2	s3	error	corr	left	right	kp	kd
count	586.0	586.000000	586.000000	586.000000	586.000000	586.000000	586.000000	586.000000	586.000000	5.860000e+02	5.860000e+02
mean	1.0	276.901024	288.054606	230.440273	139.906143	0.643157	0.019539	168.822526	117.887372	3.000000e-02	2.000000e-02
std	0.0	353.566624	343.407202	337.305607	164.801097	1.163601	0.036570	60.258234	100.736795	3.472411e-10	3.472411e-18
min	1.0	0.000000	0.000000	0.000000	0.000000	-2.790000	-0.090000	-40.000000	-40.000000	3.000000e-02	2.000000e-02
25%	1.0	37.000000	34.000000	25.000000	45.000000	0.000000	0.000000	179.000000	-40.000000	3.000000e-02	2.000000e-02
50%	1.0	60.000000	63.500000	31.000000	95.000000	0.000000	0.010000	179.000000	180.000000	3.000000e-02	2.000000e-02
75%	1.0	519.000000	652.500000	442.500000	174.000000	1.317500	0.048000	200.000000	180.000000	3.000000e-02	2.000000e-02
max	1.0	982.000000	911.000000	955.000000	944.000000	3.000000	0.090000	200.000000	200.000000	3.000000e-02	2.000000e-02



Tabla 8: Distribución de estados

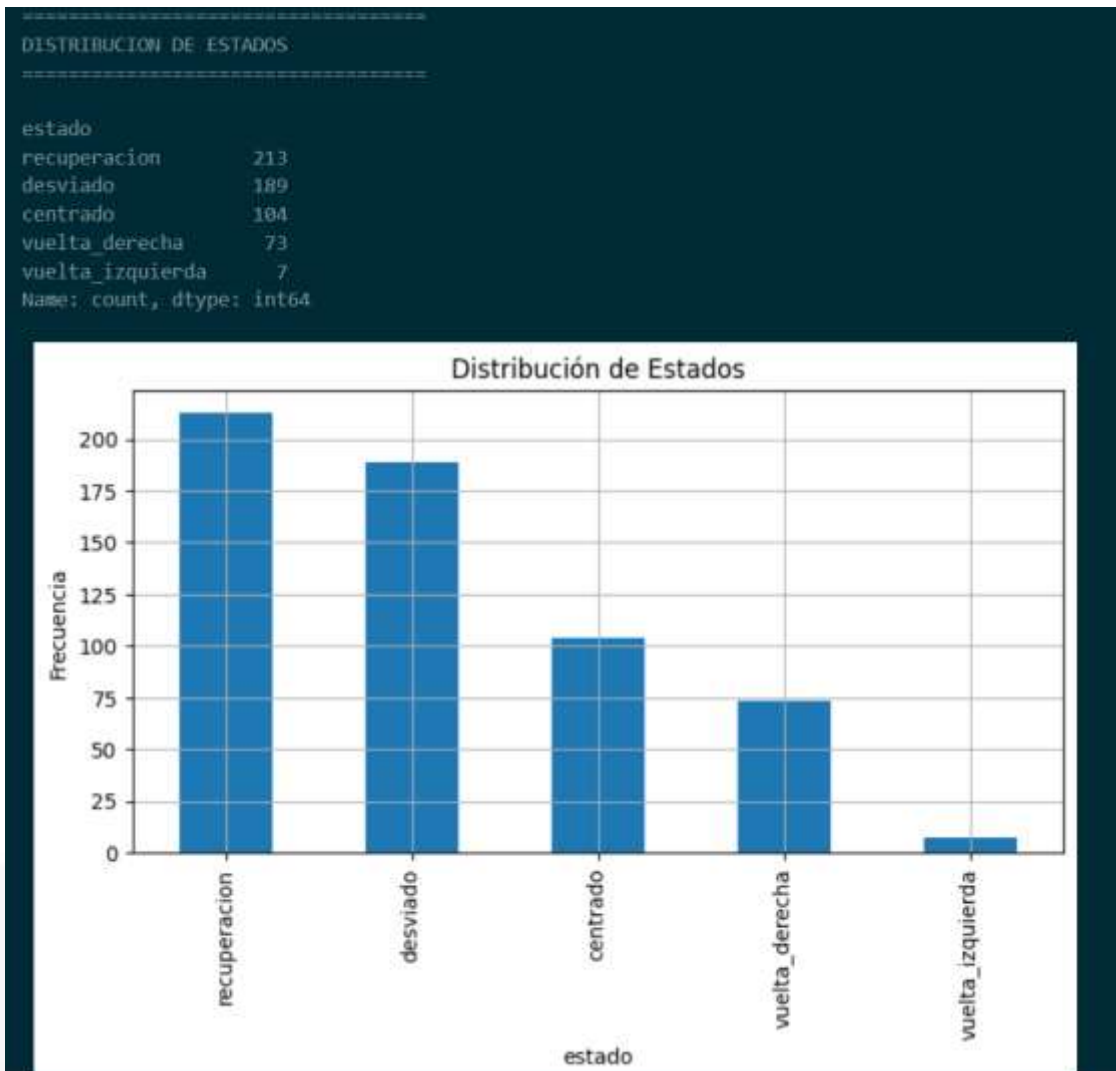




Tabla 9: Variables registradas y unidades

=====			
VARIABLES REGISTRADAS Y UNIDADES			
=====			
Variable		Descripcion	Unidad
0	hora	Hora de captura	hh:mm:ss
1	test	Identificador de prueba	Int
2	s0	Sensor infrarrojo izquierdo exterior	ADC
3	s1	Sensor infrarrojo izquierdo interior	ADC
4	s2	Sensor infrarrojo derecho interior	ADC
5	s3	Sensor infrarrojo derecho exterior	ADC
6	error	Error de posicion respecto a la linea	Float
7	corr	Correccion calculada por el controlador PD	PWM
8	left	Velocidad motor izquierdo	PWM
9	right	Velocidad motor derecho	PWM
10	kp	Ganancia proporcional	Float
11	kd	Ganancia derivativa	Float
12	estado	Estado operativo del robot	String

Descripción del modelo usado

Para el análisis del comportamiento del robot seguidor de línea se utilizó un modelo de aprendizaje automático supervisado tipo **Random Forest Classifier**. Este modelo está basado en un conjunto de árboles de decisión que trabajan en conjunto para mejorar la precisión de la clasificación y reducir el sobreajuste, lo que lo hace adecuado para datos ruidosos provenientes de sensores físicos.

Variable objetivo

La variable objetivo del modelo corresponde al campo:



estado

Esta variable representa la condición operativa del robot durante su desplazamiento y se obtiene directamente de la lógica de control implementada en el sistema embebido. Las clases consideradas incluyen estados como:

- centrado
- desviado
- vuelta_derecha
- vuelta_izquierda
- recuperacion
- fuera_linea

Estas etiquetas describen el comportamiento dinámico del robot respecto a la línea de seguimiento. Estos estados son calculados con las lecturas de los sensores y el error de esa manera determinando cual es el estado del robot seguidor de línea.

Variables de entrada

El modelo fue entrenado utilizando variables obtenidas a partir de la telemetría enviada por el ESP32 durante la operación del robot. Las principales variables de entrada fueron:

- **s0, s1, s2, s3:** lecturas normalizadas de los sensores infrarrojos QTR.
- **error:** desviación del robot respecto a la línea central.
- **corr:** corrección generada por el controlador PID.
- **left:** velocidad aplicada al motor izquierdo.
- **right:** velocidad aplicada al motor derecho.



- **kp:** ganancia proporcional del controlador.
- **kd:** ganancia derivativa del controlador.

Estas variables permiten capturar tanto el estado del entorno (línea detectada) como la respuesta del sistema de control.

Métricas principales

El desempeño del modelo fue evaluado mediante métricas estándar de clasificación, con el objetivo de validar su capacidad para identificar correctamente los estados del robot. Las métricas utilizadas fueron:

- **Accuracy (exactitud):** mide el porcentaje de predicciones correctas sobre el total de muestras.
- **Precision:** evalúa la proporción de predicciones correctas dentro de cada clase.
- **Recall:** mide la capacidad del modelo para detectar correctamente cada estado.
- **F1-Score:** combina precisión y recall en una sola métrica balanceada.
- **Matriz de confusión:** permite visualizar los aciertos y errores de clasificación entre las diferentes clases de estado.

Estas métricas permiten asegurar la validez del modelo dentro del gemelo digital y su capacidad para representar el comportamiento real del sistema físico.



Código para entrenamiento

```
python > entrenador_seguidor.py > ...
154
155 zona_muerta_95 = np.percentile(
156     errores_estables,
157     95
158 )
159
160 print("\n=====")
161 print("ZONA MUERTA RECOMENDADA")
162 print("=====")
163
164 print(
165     f"Conservadora (80%): ±{zona_muerta_80:.3f}"
166 )
167
168 print(
169     f"Recomendada (90%): ±{zona_muerta_90:.3f}"
170 )
171
172 print(
173     f"Agresiva (95%): ±{zona_muerta_95:.3f}"
174 )
175
176 # =====
177 # 10. GUARDAR MODELO
178 # =====
179
180 import joblib
181
182 joblib.dump(
183     modelo,
184     "modelo_oscilacion.pkl"
185 )
186
187 print(
188     "\nModelo guardado como:"
189     " modelo_oscilacion.pkl"
190 )
191
192 # =====
```

Figura 7: zona muerta y creación de archivo pkl del código entrenador



Resultado del entrenamiento

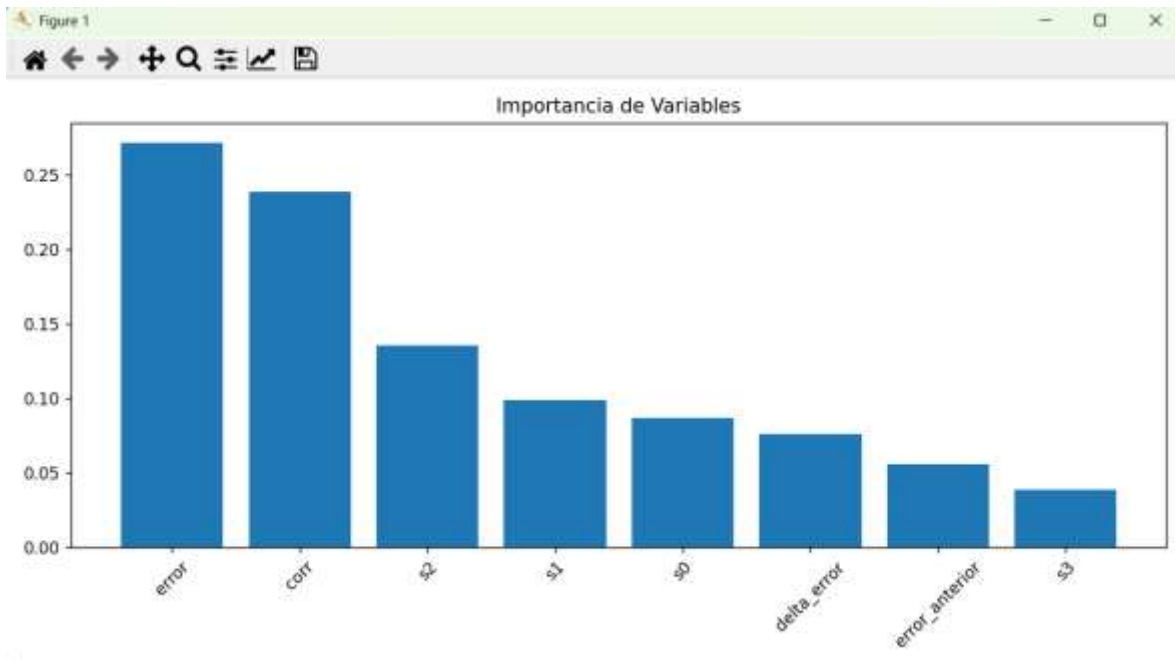


Figura 8: Grafica de importancia de las variables para el entrenamiento

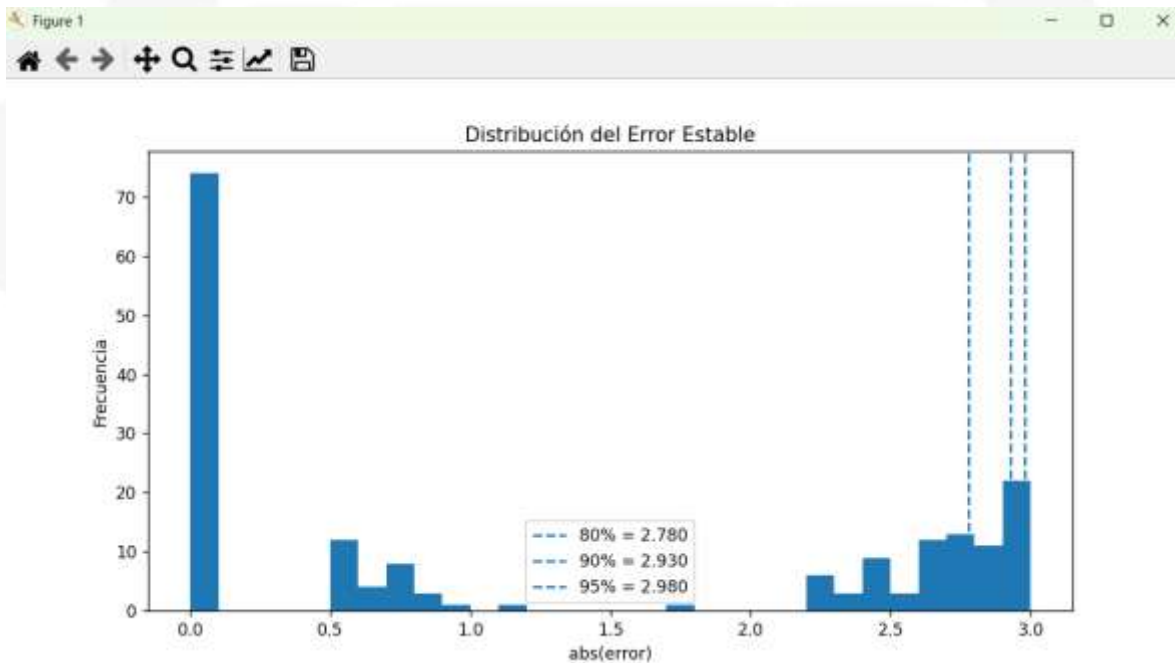


Figura 9: Tabla de dispersión del error estable



```
PS C:\Users\diego\Documents\PlatformIO\Projects\python> & C:\Users\diego\AppData\Local\Python\pyt  
r_seguidor.py  
  
Primeras filas:  
      hora test  s0  s1  s2  s3 error corr left right  kp  kd      estado  
0 11:59:36   1 916  33  18 143 0.75 0.03 179  180 0.03 0.02 centrado  
1 11:59:36   1 942  27  23  23 2.77 0.09 179  180 0.03 0.02 vuelta_derecha  
2 11:59:36   1  55 855 843  24 0.88 0.02 179  180 0.03 0.02 desviado  
3 11:59:36   1  36  43 597  38 0.00 0.00 -40  200 0.03 0.02 recuperacion  
4 11:59:37   1  39  47 431  72 0.00 0.00 -40  200 0.03 0.02 recuperacion  
  
Distribución:  
oscilando  
1    402  
0    183  
Name: count, dtype: int64  
  
=====
```

RESULTADOS DEL MODELO

=====

Precisión: 94.87%

Reporte:

Figura 10: Variables y distribución de oscilaciones

```
Reporte:  
  
      precision    recall  f1-score   support  
  
      0       0.90      0.95      0.92         37  
      1       0.97      0.95      0.96         80  
  
   accuracy                   0.95        117  
  macro avg       0.94      0.95      0.94        117  
weighted avg       0.95      0.95      0.95        117  
  
Matriz de confusión:  
[[35  2]  
 [ 4 76]]  
  
Importancia de variables:  
      Variable  Importancia  
4      error    0.271455  
5      corr     0.238404  
2      s2       0.135491  
1      s1       0.098368  
0      s0       0.086412  
7  delta_error  0.075765  
6  error_anterior 0.055741  
3      s3       0.038366
```

Figura 11: Reporte de entrenamiento, f1-score, matriz de confusión e importancia de variables



```
=====
ZONA MUERTA RECOMENDADA
=====
Conservadora (80%): ±2.780
Recomendada (90%): ±2.930
Agresiva (95%): ±2.980

Modelo guardado como: modelo_oscilacion.pkl

=====
RESUMEN FINAL
=====
Precisión del modelo: 94.87%
Zona muerta sugerida: ±2.930

Si abs(error) es menor que la zona muerta, puedes probar:

if(abs(error) < 2.930)
{
    correccion = 0;
}
PS C:\Users\diego\Documents\PlatformIO\Projects\python> |
```

Figura 12: Zona muerta recomendada, localización del archivo.pkl y resumen final del entrenamiento

Tiempo de recorrido con zona muerta

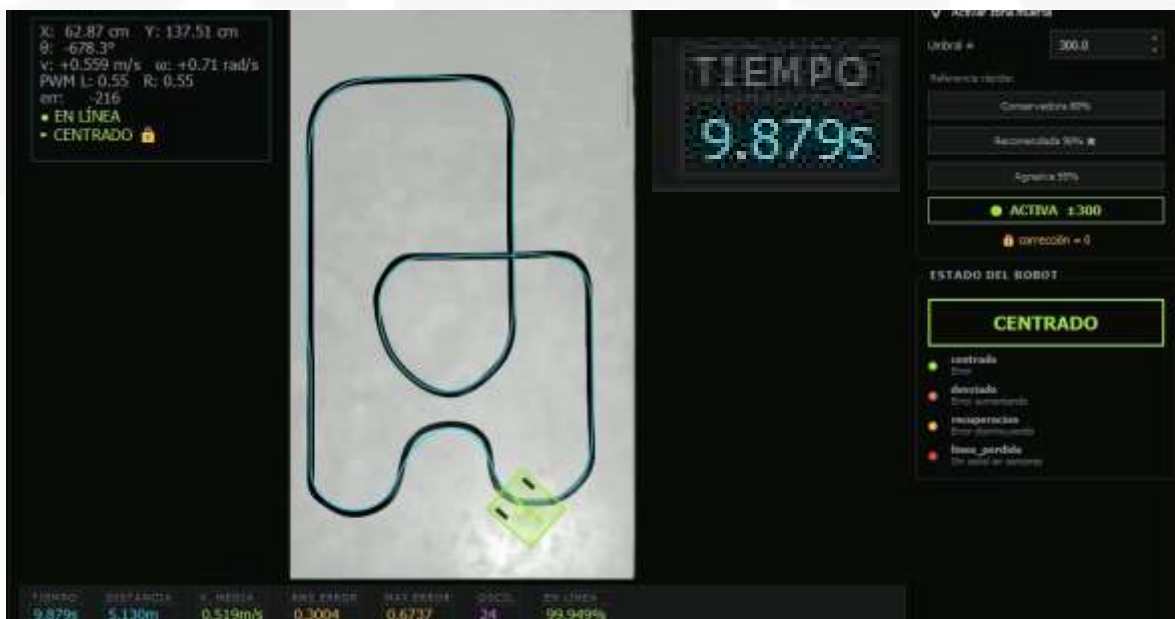


Figura 13: Tiempo de recorrido del gemelo digital con zona muerta

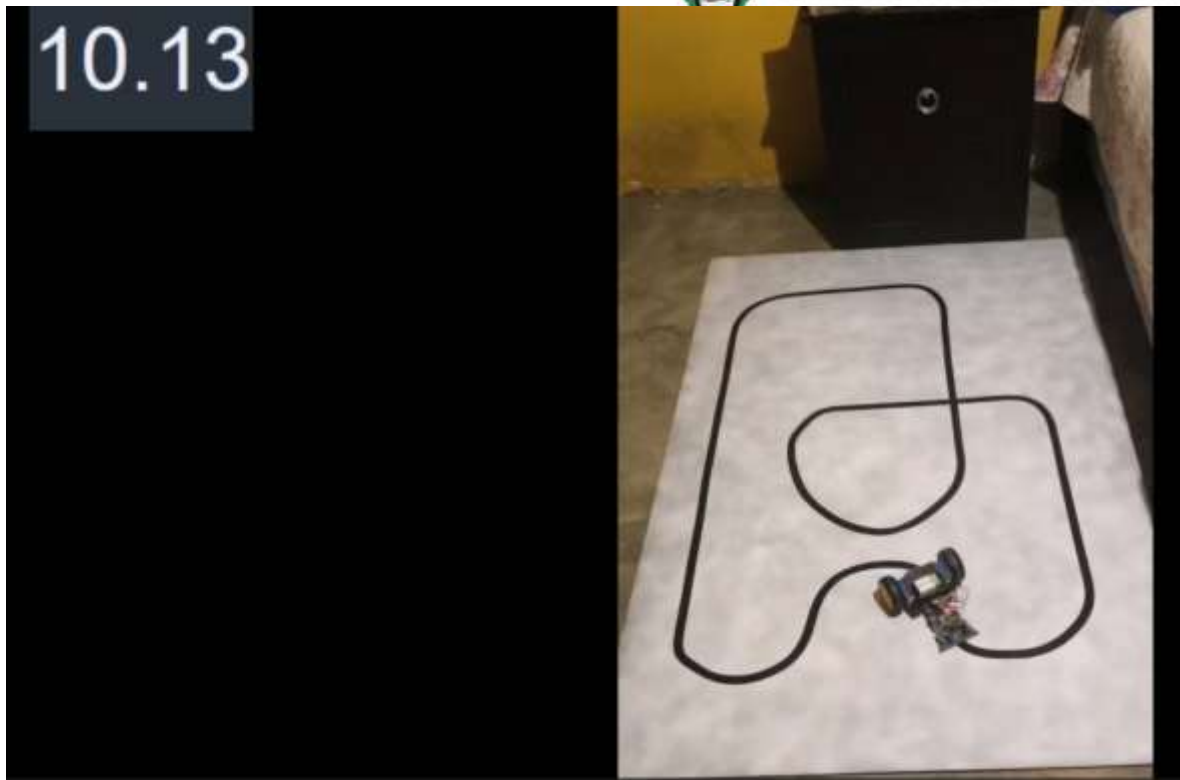


Figura 14: Tiempo de recorrido del seguidor con zona muerta activada

Correcciones echas al gemelo digital

Chasis

La configuración en cuanto a dimensiones y peso del robot se pusieron igual que el seguidor físico, incluyendo la distancia entre sensores y distancia al eje, los parámetros de fricción no se tocaron para nada y se dejaron tal cual estaba, solo se posiciono el robot en la pista.

Motor y sensores

En cuanto a los motores se subió el voltaje máximo y el nominal cerca de .20V para acercarse más al tiempo del seguidor físico, también se subieron el rpm a 330 y el torque se dejó igual incluyendo los parámetros de los sensores que no se cambiaron.



PID

Por último, el PID se puso igual que el seguidor físico, obteniendo un resultado exitoso en cuanto al comportamiento del seguidor, obteniendo un tiempo con diferencia de milésimas.

Diferencias de lógica

El programa utilizado para el gemelo digital es la modificación de un código de github por lo que la lógica principal se tuvo que mantener para el correcto funcionamiento, dando como problema que el error esta invertido, dando valores de signo contrario en comparación al seguidor de línea físico.

Tabla comparativa

Parámetro	Robot físico (real)	Gemelo digital
Condición de la pista	Pista de mdf	Archivo JPG
Condición de luz	Luz amarilla	Contraste +23
Voltaje inicial	9.18V	9.40V
Error medio	0.9308%	900.2227%
Promedio de oscilación	25.02%	22.36%
Tiempo de vuelta	10.13 segundos	9.879 segundos
Velocidad máxima	330 rpm	380 rpm
Velocidad estándar (baseSpeed)	180 pwm	255 pwm
Configuración PID (Kp, Kd)	KP=0.03	KP=0.04



	KD=0.02	KD=0.02
Configuración de zona muerta	(+/-) 2.930	(+/-) 3.00
Rango de valores sensores (s0–s3)	0 - 1000	0 - 1000

Salida del gemelo digital

Como salida del gemelo se eligió el CSV de la telemetría del mismo, con el fin de comparar ambos resultados con las mejoras sugeridas por el modelo entrenado, aunque la lógica de control es diferente de igual manera nos permite estandarizar estados conforme al error y lectura de sensores, pero para esto antes hay que hacer pruebas hasta que el tiempo de recorrido sean lo mas parecidos posible, de esta manera las modificaciones hechas en ambos modelos deberían de tener cambios similares.



Comparación de CSV de robot físico y gemelo digital

comparación de estados

— CSV 1 — Telemetría —		
	conteo	porcentaje
estado		
desviado	1089	49.91
recuperacion	1056	48.40
centrado	37	1.70

— CSV 2 — Robot físico —		
	conteo	porcentaje
estado		
recuperacion	213	36.35
desviado	189	32.25
centrado	104	17.75
vuelta_derecha	73	12.46
vuelta_izquierda	7	1.19

Figura 15: Comparación de estados



Figura 16: Grafica distribución de estados



Análisis del error

```
— Error — CSV 1 — Telemetría —  
Media:      -765.7808  
Std:        559.4169  
Min:        -1829.8052  
Max:        1452.6392  
|error| medio: 900.2227  
% muestras |error| < 1: 0.4%  
% muestras |error| < 5: 0.5%  
  
— Error — CSV 2 — Robot físico —  
Media:      0.6432  
Std:        1.1636  
Min:        -2.7900  
Max:        3.0000  
|error| medio: 0.9308  
% muestras |error| < 1: 63.5%  
% muestras |error| < 5: 100.0%
```

Figura 17: Análisis del error en Notebook

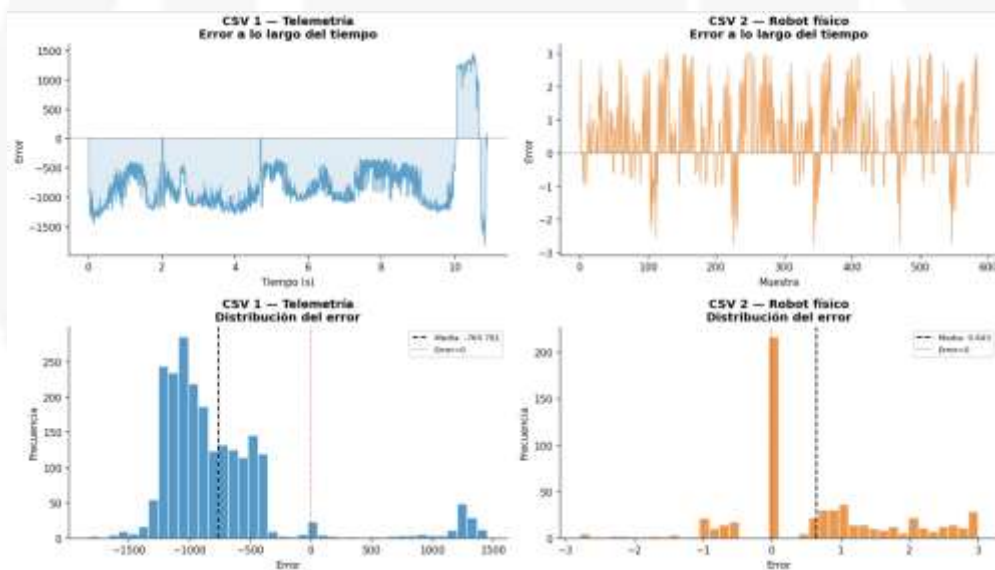


Figura 18: histograma del error

Lecciones aprendidas

Las lecciones aprendidas fueron bastantes, primeramente surgio el detalle de que al guardar el CSV no se agurdaban los nombres de las columnas, solo si se



reiniciaba por completo node—red, también vimos que cualquier cambio en el PID afectaba mucho el desempeño del robot, y que la oscilación era por el acomodo de los sensores QTR, también notamos que en el entorno virtual a pesar de tener exactamente los mismos datos, el comportamiento era notablemente diferente, por lo que se le tuvieron que hacer algunos ajustes para que el comportamiento fuera lo mas similar posible, al final quedo una diferencia de aproximadamente 1.3 segundos, ya que si subíamos mas los parámetros por alguna extraña razón el seguidor digital se salia de la pista, asi que se tuvo que dejar con esa diferencia de tiempo.

Ademas de añadir que el seguidor digital tiene el error invertido, presentando un error muy similar, pero de signo contrario, como se puede apreciar en la **Figura 18**.

Link del video

<https://drive.google.com/file/d/15APf6mwH8TxOMsd8a-gml73Nw6PMoLFE/view?usp=sharing>

Conclusiones

El proyecto final logro recopilar todo lo que se ha aprendido dentro de la materia de Ciberfisica, desde la recolección de datos hasta la simulación desarrollada mediante el gemelo digital, la cual logró reproducir de manera aceptable el comportamiento general del robot seguidor de línea, especialmente en condiciones controladas como tramos rectos y curvas suaves. Se observó que, al ajustar parámetros como la velocidad base, las ganancias del controlador y la separación de sensores, la trayectoria simulada presentó una tendencia similar a la del robot real, manteniendo estabilidad y corrigiendo desviaciones de forma comparable. Asimismo, variables como el error y la respuesta del control mostraron patrones coherentes entre ambos sistemas, lo que indica que el modelo implementado en Python representa adecuadamente la lógica de control y una aproximación razonable de la dinámica del robot.



No obstante, se identificaron ciertas diferencias importantes. El robot real presentó más ruido en las lecturas de los sensores y pequeñas variaciones mecánicas (como fricción irregular, desbalance en motores o variaciones en la superficie) que no están completamente representadas en la simulación. Además, el modelo virtual tiende a ser más “suave” y predecible, mientras que el robot físico puede mostrar oscilaciones más abruptas, especialmente en curvas cerradas o al perder la línea. También se observó que el sistema real responde de forma distinta ante cambios bruscos de velocidad, debido a factores como la inercia real y las limitaciones eléctricas del hardware controlado por el ESP32.

El entrenamiento del modelo ayudo a reducir significativamente la oscilación del seguidor de línea físico logrando asi reducir su tiempo y dando la posibilidad de subir la velocidad para un menor tiempo de recorrido en la pista.

Referencias

- Banks, A., & Gupta, R. (2014). *MQTT Version 3.1.1*. OASIS.
- Hunt, V. D. (2013). *Industrial Automation: Hands On*. McGraw-Hill.
- Rajkumar, R. (2010). *Cyber-Physical Systems*. IEEE.
- Node-RED Documentation.
- Eclipse Mosquitto Documentation.